

# Project Workflow

## 1. Core Execution Engine

The backend relies on an active instance of a FastAPI application initialized inside Main.py. Static interface styling assets are mapped dynamically using FastAPI's native mounting tool, while a template renderer maps the primary view engine onto the client viewport. Security guardrails execute immediately upon runtime initialization by scanning the local environment file for a valid, non-OAuth Google API credential to prevent unauthenticated execution states.

## 2. Workflow Breakdown

**Step 1: Input Submission:** The client opens the portal view and picks an objective from a drop-down menu. They type their topic or query into a text area element and trigger an HTTP POST request targeted at the /generate backend route.

**Step 2: Task Extraction and Selection:** The FastAPI controller reads the payload, verifying the core string values and identifying the execution enum class. A conditional branching block assigns the processing duties to a specific standalone task module.

**Step 3: Route A - Local ML Execution:** If the user selects concept explanation, the platform diverts processing to the local explanation script. This script loads a Seq2Seq tokenizer and model weights into memory, processes the string into hardware tokens, runs execution directly on local CPU cores, and decodes the result back into raw text.

**Step 4: Route B - Cloud AI Generation:** For all other operations, the application invokes the Google GenAI SDK. It passes the active client configuration along with custom prompt blueprints directly to remote API infrastructure for quick completion.

**Step 5: View Compiling:** The generated textual results are piped back to the main app loop. If an exception occurs, a generic handler catches the error message and labels the active model as a system failure. The data binds directly into the interface markup through the templating layer, displaying a clean response card to the final consumer.

